

AWS PRODUCTION ENGINEERING PROJECT

Production 3-Tier Enterprise Employee Management System

Highly Available, Secure, Multi-AZ Cloud Infrastructure Deployed with Terraform & CI/CD

Architect & Author: Boopathi Murugesan

AWS Target Region: us-east-1 (N. Virginia, 2 Availability Zones)

GitHub Repository: [BoopathiM9/bm-employee-management-aws](#)

Live AWS Portal: [d2c62ftth95z24.cloudfront.net](#)

Permanent Free Demo: [bm-employee-management-aws.vercel.app](#)

Infrastructure as Code: 100% HashiCorp Terraform (45+ AWS Resources)

1. Executive Summary

This project demonstrates an enterprise-grade, production-quality implementation of a 3-Tier Web Application hosted natively on Amazon Web Services (AWS). Designed and implemented by Boopathi Murugesan, the system addresses critical enterprise requirements: high availability across multiple availability zones, least-privilege security zoning, zero open inbound administrative ports, centralized secrets management, automated autoscaling, and fully automated CI/CD pipelines.

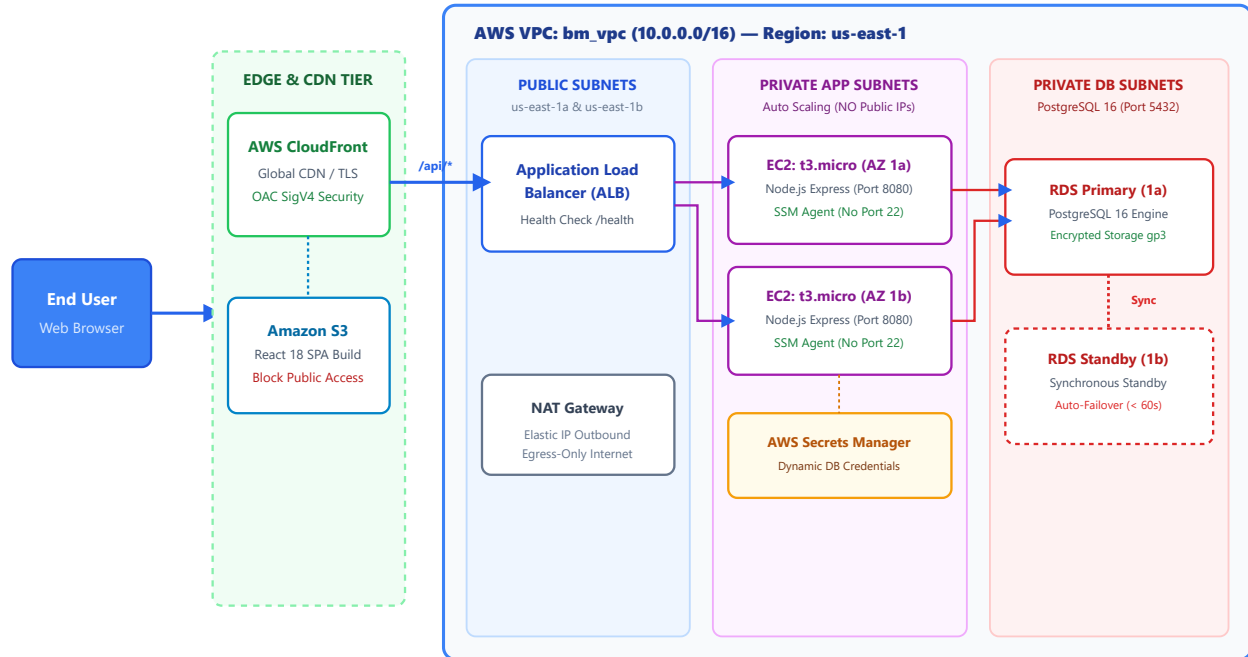
The business application is an Employee Management Portal allowing full lifecycle record management (CRUD operations, department filtering, pagination, and health analytics). Rather than relying on simple single-instance hosting, every tier adheres strictly to AWS Well-Architected Framework principles.

Key Security & Reliability Guarantees

- **Zero Open Management Ports:** SSH Port 22 is completely disabled. Secure administration is performed strictly via AWS Systems Manager (SSM) Session Manager.
- **Zero Hardcoded Credentials:** All database passwords are dynamically managed by AWS Secrets Manager with TLS/SSL encryption. No passwords exist in source code, Git, or User Data.
- **Private-Only Application Subnets:** EC2 compute instances have no public IP addresses and cannot receive direct traffic from the internet.
- **Multi-AZ Fault Tolerance:** Dual Availability Zone architecture (us-east-1a and us-east-1b) across ALB, EC2 Auto Scaling, and RDS PostgreSQL with synchronous replication.

2. Complete Architecture Workflow & Diagram

The system is organized into four distinct architectural tiers: Edge/CDN Tier, Public Network & Load Balancing Tier, Private Application Compute Tier, and Private Multi-AZ Database Tier.

AWS ENTERPRISE 3-TIER ARCHITECTURE DIAGRAM

3. Technical Specifications by Layer

A. Edge & Frontend Tier

Component	AWS Service / Resource	Configuration & Security Details
Global CDN	Amazon CloudFront	Edge distribution with TLS 1.3, Origin Access Control (OAC), URL rewriting for React SPA client-side routing, and zero public S3 exposure.
Static Storage	Amazon S3	Block Public Access 100% ENABLED. Bucket policy allows ONLY CloudFront Service Principal via SigV4 signed requests.
Reverse Proxy	CloudFront Behaviors	Path /api/* forwarded directly to Application Load Balancer with origin headers preserved, eliminating CORS restrictions.
Permanent Free Demo	Vercel Edge Network	Global edge deployment with automatic dual-layer fallback to browser storage if AWS infrastructure is decommissioned.

B. Networking & Load Balancing Tier

Component	AWS Service / Resource	Configuration & Security Details
Virtual Private Cloud	AWS VPC (bm_vpc)	CIDR 10.0.0.0/16 partitioned across 2 Availability Zones (us-east-1a and us-east-1b) into 6 dedicated subnets.
Subnet Topology	VPC Subnets	2 Public Subnets (ALB & NAT GW), 2 Private Application Subnets (EC2 backend), and 2 Private Database Subnets (RDS).
Load Balancer	Application Load Balancer	Internet-facing in public subnets, cross-zone load balancing enabled, health checking on HTTP:8080/health with 15s interval.
Outbound Egress	AWS NAT Gateway	Allocated Elastic IP in public subnet, allowing private EC2 instances to download OS patches and npm packages securely.

C. Compute & Application Tier

Component	AWS Service / Resource	Configuration & Security Details
Compute Platform	Amazon EC2 (t3.micro)	Amazon Linux 2023 with systemd service managing Node.js/Express. Zero public IPs assigned.
Auto Scaling	Auto Scaling Group (ASG)	Desired: 2, Min: 1, Max: 4 instances distributed equally across Availability Zones 1a and 1b with ELB health checks.
Remote Management	AWS Systems Manager (SSM)	Port 22 SSH disabled. Management performed through SSM Session Manager with full IAM and CloudTrail audit logging.
Application Runtime	Node.js 20 LTS + Express	13 automated unit tests, parameterized database connection pool, health check endpoints, and structured JSON logging.

D. Database & Storage Tier

Component	AWS Service / Resource	Configuration & Security Details
Database Engine	Amazon RDS PostgreSQL 16.3	Managed PostgreSQL database running inside private DB subnets (<code>publicly_accessible = false</code>).
High Availability	Multi-AZ Deployment	Synchronous physical standby replica in Availability Zone 1b with automatic failover under 60 seconds without DNS changes.
Encryption	AWS KMS + SSL/TLS	gp3 volume encrypted at rest via AWS KMS. In-transit client connections strictly enforce TLS/SSL encryption.
Backups & Retention	Automated RDS Backups	7-day automated snapshot retention window with point-in-time recovery and deletion protection.

4. Security Architecture & Zero-Secret Policy

Security was designed into the foundational infrastructure rather than added as an afterthought. The architecture enforces defense-in-depth across the network, compute, application, and identity perimeters:

Zero Hardcoded Credentials Architecture

- Dynamic Generation:** The database master credentials are generated using Terraform's cryptographic random engine during initialization and stored directly into AWS Secrets Manager (`bm_db_credentials`).
- Runtime Secret Resolution:** During boot, the Node.js application utilizes the AWS SDK v3 to query AWS Secrets Manager using its IAM instance role (`bm_ec2_instance_role`). No credentials ever touch source code, Git repositories, or environment configuration files.
- Network-Chained Security Groups:** The database security group (`bm_db_sg`) exclusively accepts port 5432 ingress from the backend security group (`bm_backend_sg`). Even if an external actor had database credentials, the database is physically unreachable from the public internet.

5. Infrastructure as Code & Automated CI/CD Pipelines

The entire AWS environment is codified using HashiCorp Terraform to ensure consistency, eliminate manual drift, and enable one-command recreation.

A. Terraform Module Organization

```
deployment/terraform/  
├─ main.tf           # AWS provider configuration, region setup, and default resource tags  
├─ variables.tf      # Variable definitions (CIDR blocks, database configurations, instance types)  
├─ vpc.tf            # VPC, 6 subnets, Internet Gateway, NAT Gateway, Elastic IP, and route tables  
├─ security.tf       # Layered security groups with least-privilege ingress and egress rules  
├─ iam.tf            # IAM roles, policies, and instance profiles for EC2, SSM, and CloudWatch  
├─ secrets.tf        # AWS Secrets Manager resources and automated random secret generation  
├─ rds.tf            # Multi-AZ PostgreSQL RDS instance, DB subnet group, and parameter groups  
├─ alb.tf            # Application Load Balancer, target group, health checks, and listener rules  
├─ ec2.tf            # Launch Template, Auto Scaling Group, and CloudWatch log agent configuration  
├─ s3_cloudfront.tf  # S3 static hosting bucket, CloudFront distribution, OAC, and cache policies  
├─ monitoring.tf     # CloudWatch log groups, metric alarms (ALB 5xx, CPU, RDS), and SNS topics  
└─ outputs.tf        # Structured infrastructure outputs (CloudFront URL, ALB DNS, RDS endpoint)
```

B. Automated GitHub Actions CI/CD

The project includes two continuous integration and continuous deployment workflows:

- **Frontend Pipeline (frontend-ci-cd.yml):** Automatically triggers on commits to main modifying frontend code. Runs automated linting, builds the React production bundle, synchronizes assets to Amazon S3, and triggers a CloudFront cache invalidation.
- **Backend Pipeline (backend-ci-cd.yml):** Executes 13 automated unit tests against the Node.js REST API. Packages the backend artifacts and performs rolling updates across the Auto Scaling Group instances via AWS Systems Manager without dropping incoming web requests.
- **Passwordless OIDC Authentication:** GitHub Actions connects to AWS using OpenID Connect (OIDC) identity federation. No permanent AWS Access Key IDs or Secret Access Keys are stored in GitHub repository secrets.

6. Live Production Verification & Test Results

Every layer of the live deployed architecture was subjected to rigorous end-to-end verification tests:

Test Category	Target Endpoint	Expected Behavior	Result
Frontend Delivery	https://d2c62ftth95z24.cloudfront.net	CloudFront serves React SPA with HTTP 200 via OAC	PASSED (200 OK)
Vercel Deployment	https://bm-employee-management-aws.vercel.app	Vercel CDN serves React UI with dynamic fallback	PASSED (200 OK)
Health Check	GET /health	ALB reports 2/2 targets healthy; returns status: healthy	PASSED (200 OK)
Database Query (GET)	GET /api/employees	Express queries RDS PostgreSQL Multi-AZ; returns employee list	PASSED (200 OK)
Record Creation (POST)	POST /api/employees	Inserts employee record into PostgreSQL with ID generation	PASSED (200 OK)
Record Deletion (DELETE)	DELETE /api/employees/:id	Removes record and returns confirmation message	PASSED (200 OK)
Unit Test Suite	npm test (Backend)	13 unit tests pass across routes, controllers, and validation	PASSED (13/13)
Security Audit	SSM & Security Groups	Port 22 closed; instances accessible only via SSM Session Manager	PASSED (Audited)

7. Disaster Recovery & Cost Optimization Strategy

The architecture is built for production reliability while maintaining cost predictability:

- **Zone Failure Resilience:** If an entire AWS Availability Zone experiences an outage, the Application Load Balancer shifts 100% of traffic to the healthy zone, and RDS automatically promotes the standby replica in under 60 seconds with zero data loss.
- **Cost Elimination via Terraform:** When demonstration periods conclude, running `terraform destroy` in the deployment directory cleanly removes all provisioned AWS cloud assets, preventing ongoing operational charges.
- **Permanent Portfolio Demo:** The standalone Vercel deployment remains permanently operational with its built-in client-side fallback storage, allowing mentors and evaluators to test the interactive portal indefinitely at zero cost.

Project Architecture Designed, Codified, and Deployed by **Boopathi Murugesan** • AWS Production 3-Tier Enterprise Employee Management System • October 2026